ESAME DI JAVASCRIPT · 13/11/2026

Cognome e nome _____ Punti _____ / 30,5

Parte 1 · Teoria (9 punti)

Una sola risposta corretta per domanda. Corretta **+0,6**, sbagliata **–0,15**, non data **0**

1. *Che cosa vale per una variabile dichiarata con* `const`*?*
   - A. Il suo valore non può cambiare in nessun modo, nemmeno le proprietà di un oggetto
   - B. Ha lo scope di funzione, come `var`
   - C. Non si può riassegnare, ma se contiene un oggetto o un array il contenuto si può modificare
   - D. Si può dichiarare senza assegnarle un valore
   - E. Nessuna delle precedenti

2. *Qual è la differenza tra* `==` *e* `===`*?*
   - A. `===` confronta valore e tipo senza conversioni, `==` può convertire i tipi prima di confrontare
   - B. Sono equivalenti, `===` è solo più leggibile
   - C. `==` confronta anche il tipo, `===` solo il valore
   - D. `===` si può usare solo con i numeri
   - E. Nessuna delle precedenti

3. *Quale di questi valori è falsy?*
   - A. `"0"`
   - B. `[]`
   - C. `"false"`
   - D. `0`
   - E. Nessuna delle precedenti

4. *Che cosa fa* `continue` *dentro un ciclo?*
   - A. Esce dal ciclo
   - B. Salta il resto dell'iterazione corrente e passa alla successiva
   - C. Ricomincia il ciclo dalla prima iterazione
   - D. Termina la funzione che contiene il ciclo
   - E. Nessuna delle precedenti

5. *Quale arrow function è equivalente a* `function doppio(x) { return x * 2 }`*?*
   - A. `const doppio = (x) => { x * 2 }`
   - B. `const doppio = (x) -> x * 2`
   - C. `const doppio => x * 2`
   - D. `const doppio = (x) => x * 2`
   - E. Nessuna delle precedenti

6. *Che cos'è una callback?*
   - A. Una funzione passata come argomento a un'altra funzione, che la chiamerà
   - B. Una funzione che chiama se stessa
   - C. Una funzione che restituisce sempre una Promise
   - D. Una funzione dichiarata dentro un ciclo
   - E. Nessuna delle precedenti

7. *Con* `const v = [10, 20, 30]`*, quanto valgono* `v.length` *e* `v[v.length - 1]`*?*
    A. `3` e `20`
    B. `2` e `30`
    C. `3` e `undefined`
    D. `4` e `30`
    E. Nessuna delle precedenti

8. *Che cosa restituisce* `[1, 2, 3, 4].filter((n) => n % 2 === 0)`*?*
    A. `[1, 3]`
    B. `[2, 4]`
    C. `[false, true, false, true]`
    D. `2`
    E. Nessuna delle precedenti

9. *A che cosa serve* `JSON.stringify`*?*
    A. Trasforma un oggetto in una stringa in formato JSON
    B. Trasforma una stringa JSON in un oggetto
    C. Crea una copia indipendente di un array
    D. Controlla che un oggetto non abbia proprietà vuote
    E. Nessuna delle precedenti

10. *Che cosa serve per usare* `import` *ed* `export` *in un file caricato da una pagina HTML?*
    A. Basta un normale `<script src="app.js">`
    B. Bisogna sostituirli con `require`
    C. Il file deve avere estensione `.json`
    D. Il tag deve essere `<script type="module" src="app.js">`
    E. Nessuna delle precedenti

11. *Che cosa restituisce* `document.querySelector(".voto")`*?*
    A. Tutti gli elementi con classe `voto`
    B. Il primo elemento con classe `voto`, oppure `null` se non ce n'è nessuno
    C. Il testo del primo elemento con classe `voto`
    D. L'elemento con id `voto`
    E. Nessuna delle precedenti

12. *Dentro una funzione* `async`*, che cosa fa* `await` *davanti a una Promise?*
    A. Blocca tutto il programma, compresi i click nella pagina, finché la Promise non è risolta
    B. Crea un nuovo thread in cui eseguire la Promise
    C. Sospende la funzione finché la Promise non è risolta e restituisce il suo valore
    D. Annulla la Promise se impiega troppo tempo
    E. Nessuna delle precedenti

13. *In un'API REST, quale verbo HTTP si usa per leggere una risorsa, per esempio l'elenco degli studenti?*
    A. `POST`
    B. `PUT`
    C. `GET`
    D. `DELETE`
    E. Nessuna delle precedenti

14. *Una richiesta riceve come risposta lo status code* `404`*. Che cosa significa?*
    A. La richiesta è andata a buon fine
    B. La risorsa è stata creata
    C. Il server ha avuto un errore interno
    D. La risorsa richiesta non esiste
    E. Nessuna delle precedenti

15. *Una Promise creata con* **`new Promise((resolve, reject) => ...)`** *chiama* **`reject("Errore ricezione dati!")`**. *Quale metodo riceve il messaggio di errore?*

    A. **`.catch()`**
    B. **`.then()`**
    C. **`.finally()`**
    D. **`resolve`**
    E. Nessuna delle precedenti

Parte 2 · Che cosa stampa (6 punti, 1,5 a esercizio)

Scrivere l'output esatto, oppure indicare che il codice genera un errore.

16. *Che cosa stampa questo codice?*

```
let x = 1;
if (true) {
  let x = 2;
  console.log(x);
}
console.log(x);
```

Output

17. *Che cosa stampa questo codice?*

```
const a = [1, 2, 3];
const b = a;
b.push(4);
console.log(a.length, b.length);
```

Output

18. *Che cosa stampa questo codice?*

```
let somma = 0;
for (let i = 1; i <= 5; i++) {
  if (i % 2 === 0) continue;
  somma += i;
}
console.log(somma);
```

Output

19. *Che cosa stampa questo codice?*

```
console.log("A");
setTimeout(() => console.log("B"), 0);
console.log("C");
```

Output

Parte 3 · Trova l'errore (4,5 punti, 1,5 a esercizio)

Indicare la riga, se l'errore è di sintassi (il programma non parte) o di esecuzione, e la correzione.

20. *Dove si trova l'errore?*

```
1 const voti = [7, 8];
2 voti.push(9);
3 voti = [10];
4 console.log(voti);
```

Riga _____

☐ Sintassi ☐ Esecuzione

Correzione

21. *Dove si trova l'errore?*

```
1 const studente = { nome: "Anna" };
2 console.log(studente.voto);
3 console.log(studente.voto.length);
```

Riga _____

☐ Sintassi ☐ Esecuzione

Correzione

22. *Dove si trova l'errore?*

```
1 function carica() {
2   const res = await fetch("/api/studenti");
3   return res.json();
4 }
```

Riga _____

☐ Sintassi ☐ Esecuzione

Correzione

Parte 4 · Listato (11 punti)

```
const studenti = [
  { nome: "Anna", voti: [8, 9, 7] },
  { nome: "Luca", voti: [5, 6] },
  { nome: "Sara", voti: [] },
];

function media(voti) {
  if (voti.length === 0) return 0;
  let somma = 0;
  for (const v of voti) {
    somma += v;
  }
  return somma / voti.length;
}

function promossi(lista) {
  return lista
    .filter((s) => media(s.voti) >= 6)
    .map((s) => s.nome);
}
```

```
function aggiungiVoto(lista, nome, voto) {
  const s = lista.find((x) => x.nome === nome);
  if (s === undefined) return false;
  s.voti.push(voto);
  return true;
}

console.log(media(studenti[0].voti));
console.log(promossi(studenti));
aggiungiVoto(studenti, "Luca", 8);
aggiungiVoto(studenti, "Marco", 10);
console.log(studenti[1].voti);
console.log(promossi(studenti));
const ok = aggiungiVoto(studenti, "Sara", 4);
console.log(ok, media(studenti[2].voti));
```

23. **Che cosa stampa (5 punti).** *Scrivere l'output del programma, una riga per ogni* `console.log`*.*

24. **Domande sul codice (3 punti).** *Rispondere in una o due righe.*

    a) *Che cosa restituisce* `aggiungiVoto(studenti, "Marco", 10)`*, e perché?*

    b) *Che cosa restituirebbe* `media([])` *senza la riga* `if (voti.length === 0) return 0`*?*

    c) `studenti` *è dichiarato con* `const`*, eppure i voti di Luca cambiano. Perché non è un errore?*

25. **Promise (3 punti).** *Che cosa stampa questo codice? Scrivere una riga per ogni* `console.log` *eseguito, nell'ordine.*

```
const datiPromise = new Promise((resolve, reject) => {
  const successo = true;
  setTimeout(() => {
    if (successo) {
      resolve("Dati ricevuti");
    } else {
      reject("Errore ricezione dati!");
    }
  }, 500);
});

console.log("Inizio");
datiPromise
  .then((valore) => console.log("Successo:", valore))
  .catch((e) => console.log("Errore:", e))
  .finally(() => console.log("Fine caricamento"));
console.log("Working...");
```

Output